

CSE 815 · Machine Learning

Department of CSE · University of Chittagong



Neural Networks in Code

TensorFlow, Forward Propagation from Scratch, and Matrix Multiplication

Rokan Uddin Faruqui

Professor, Department of CSE

University of Chittagong

`rufaruqui@cu.ac.bd`

Week 4 · Part 2

Part 1 gave us

$$a_j^{[l]} = g(\mathbf{w}_j^{[l]} \cdot \mathbf{a}^{[l-1]} + b_j^{[l]}), \quad \mathbf{a}^{[0]} = \mathbf{x}$$

and forward propagation: apply it layer by layer, left to right.

Today we make that equation run — three times. First with a library, so you see how little code a practitioner writes; then from scratch in NumPy, so you know what the library does; then as one matrix product, which is why it runs fast.

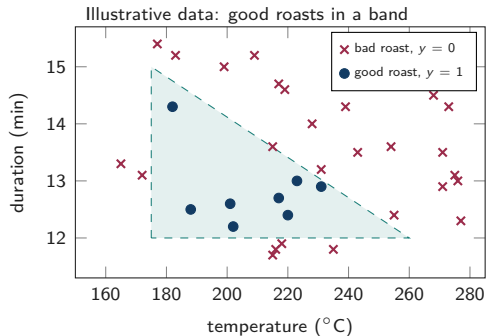
Today

1. Inference in code: the coffee-roasting network
2. Data in TensorFlow: shapes and tensors
3. Building a network with `Sequential`
4. Forward propagation from scratch in NumPy
5. Is there a path to AGI?
6. Vectorisation and matrix multiplication

The running example is the $2 \rightarrow 3 \rightarrow 1$ network of Part 1, so every number can be checked against the hand calculation.

Inference in Code

Example: roasting coffee beans



The task

Inputs: x_1 = temperature, x_2 = duration. Too cool or too short: undercooked. Too hot or too long: burnt. Good roasts lie in the triangle.

Why one neuron is not enough

The good region is bounded by *three* lines. A single logistic unit draws one. A hidden layer of three units can learn one line each, and the output unit combines them.

Network: $2 \rightarrow 3 \rightarrow 1$, sigmoid everywhere.

Forward propagation in TensorFlow

```
import numpy as np
from tensorflow.keras.layers import Dense

x = np.array([[200.0, 17.0]])    # one example

layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)                  # shape (1, 3)

layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)                 # shape (1, 1)

yhat = 1 if a2 >= 0.5 else 0
```

Each line is one row of the forward-propagation equations: build a layer, apply it to the previous activations.

What Dense means

A **dense** (fully connected) layer: every unit receives every activation from the previous layer. `units` sets the layer's width; `activation` sets g .

Where are w and b ?

Inside the layer object. They are created — with random values — the first time the layer sees an input, because only then is n_{in} known. Training (Week 5) replaces them with good values.

Scale the inputs

200 and 17 differ by an order of magnitude. Normalise features before training, exactly as in Week 2.

The same pattern for handwritten digits

```
x = np.array([[0.0, ..., 245, ..., 240, ..., 0]])  
                                     # shape (1, 64)  
layer_1 = Dense(units=25, activation='sigmoid')  
a1 = layer_1(x)  
layer_2 = Dense(units=15, activation='sigmoid')  
a2 = layer_2(a1)  
layer_3 = Dense(units=1, activation='sigmoid')  
a3 = layer_3(a2)  
  
yhat = 1 if a3 >= 0.5 else 0
```

Read it against Part 1

64 → 25 → 15 → 1: three Dense layers, one per layer of the diagram. The input row holds the 64 pixel brightnesses.

Key idea

Writing the network one layer at a time is verbose but transparent: you can inspect `a1` and `a2` — the learned features — directly. That is how the “edges, then parts” pictures of Part 1 are made.

Data in TensorFlow

One example is a $1 \times n$ matrix, not a vector

```
np.array([200.0, 17.0])      # shape (2,)
np.array([[200.0, 17.0]])    # shape (1, 2) row
np.array([[200.0], [17.0]])  # shape (2, 1) column
```

Python	shape	meaning
[200, 17]	(2,)	1-D array: no rows or columns
[[200, 17]]	(1, 2)	one example, two features
[[200], [17]]	(2, 1)	two examples, one feature

Convention used by TensorFlow

A batch of data is a 2-D matrix with **one row per example** and **one column per feature**: shape (m, n) . A single example is therefore $(1, n)$.

1-D arrays

Code in Weeks 1–3 used 1-D arrays such as `np.array([1, 2])`. TensorFlow layers expect 2-D input; passing shape $(2,)$ raises an error or, worse, is broadcast into something you did not mean. Print `.shape` whenever in doubt.

```
a1 = layer_1(x)
print(a1)
# tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3),
#           dtype=float32)

a1.numpy()           # back to a NumPy array
# array([[0.2, 0.7, 0.3]], dtype=float32)
```

Values shown are placeholders: an untrained layer has random weights, so your numbers will differ.

What a tensor is

TensorFlow's own array type, `tf.Tensor`: a matrix (or higher-dimensional array) that can live on a GPU and remember how it was computed, for training.

Living with two types

NumPy came first; TensorFlow converts NumPy input automatically, and returns tensors. Call `.numpy()` when you want to plot or print an activation with ordinary tools.

Key idea

The *data* is the same in both. Only the container differs — and the (m, n) shape convention holds in both.

Building a Neural Network

Sequential: string the layers together

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=3, activation='sigmoid'),
    Dense(units=1, activation='sigmoid')])

X = np.array([[200.0, 17.0],
              [120.0,  5.0],
              [425.0, 20.0],
              [212.0, 18.0]])    # shape (4, 2)
y = np.array([1, 0, 0, 1])

model.compile(...)              # Week 5
model.fit(X, y)                  # Week 5
model.predict(X_new)             # forward prop
```

What changed

Sequential chains the layers so that `model.predict` runs the whole forward propagation for you — on every row of `X_new` at once.

Digits, in one statement

```
model = Sequential([
    Dense(units=25, activation='sigmoid'),
    Dense(units=15, activation='sigmoid'),
    Dense(units=1,  activation='sigmoid')])
```

Do not stop at the library

Five lines train a network. When it misbehaves, only someone who knows what those lines compute can debug it. Hence the next section.

Forward Propagation from Scratch

```
def g(z):  
    return 1 / (1 + np.exp(-z))  
  
x = np.array([1, 2])  
  
w1_1 = np.array([1, 2]); b1_1 = -1  
z1_1 = np.dot(w1_1, x) + b1_1  
a1_1 = g(z1_1) # 0.9820  
  
w1_2 = np.array([-3, 4]); b1_2 = 1  
a1_2 = g(np.dot(w1_2, x) + b1_2) # 0.9975  
  
w1_3 = np.array([5, -6]); b1_3 = 2  
a1_3 = g(np.dot(w1_3, x) + b1_3) # 0.0067  
  
a1 = np.array([a1_1, a1_2, a1_3])
```

Naming

$w_{1,2}$ stands for $w_2^{[1]}$: layer 1, unit 2. The code is the Part 1 table, line for line — and produces the same three numbers.

What repeats

Each unit: dot product, add bias, apply g . Only the parameters change. That repetition is a loop waiting to be written — the next slide.

Layer 2 is the same three lines again, with $w_1^{[2]} = [2, -1, -3]$, $b_1^{[2]} = 0.5$ and input a_1 : it returns 0.8094.

```
def dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]          # column j = unit j
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out

W1 = np.array([[1, -3, 5],
               [2, 4, -6]])  # (n_in, units)
b1 = np.array([-1, 1, 2])
W2 = np.array([[2], [-1], [-3]])
b2 = np.array([0.5])
def sequential(x):
    a1 = dense(x, W1, b1)
    a2 = dense(a1, W2, b2)
    return a2
sequential(np.array([1, 2]))  # [0.8094]
sequential(np.array([-1, -1]))  # [0.0562]
```

Stacking parameters into W

Put each unit's weight vector in a **column**:

$$\mathbf{W}^{[1]} = \begin{bmatrix} \mathbf{w}_1^{[1]} & \mathbf{w}_2^{[1]} & \mathbf{w}_3^{[1]} \end{bmatrix} = \begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix}$$

Shape $(n_{in}, \text{units}) = (2, 3)$.

Key idea

The function `dense` implements one layer of any width, and `sequential` chains layers. Together they are a complete inference engine — what `model.predict` does, minus speed. Both outputs match Part 1.

Is There a Path to AGI?

Two different things called “AI”

ANI — artificial *narrow* intelligence: a system that does one task, such as spam filtering, crop-disease detection or route planning. Enormous progress, enormous economic value.

AGI — artificial *general* intelligence: a system that can do anything a typical human can.

Progress in ANI is real and measurable. It does not, by itself, imply progress towards AGI, though the two are often conflated in public discussion.

Why the neuron analogy does not settle it

- Our unit $g(\mathbf{w} \cdot \mathbf{x} + b)$ is a drastic simplification of a biological neuron.
- We still do not know, in any detail, how the brain learns.
- So “simulate enough neurons” is not, today, a recipe for intelligence.

Be precise in claims

“The model does task T at level L on data D ” is a scientific claim. “The model understands” is not, until you define and test it.

Neural rewiring

In experiments on newborn ferrets, visual input was rerouted to the brain region that normally processes sound. That region developed responses resembling those of the visual cortex: it learned to “see”.

Sensory substitution in people

- A camera driving a grid of electrodes on the tongue lets blind users perceive shapes.
- Some people learn to locate objects by echolocation (clicks).
- A belt that buzzes towards north gives a sense of direction.

The hypothesis

Much of the brain may run one (or a few) general learning algorithms, which become specialised by the data they receive.

If so, finding a close enough approximation of that algorithm would be a path to AGI. That is a hope, not a result: nobody knows whether it is true, or what the algorithm is.

For this course, the useful lesson is narrower and certain: the *same* architecture — a stack of dense layers — learns edges from pixels, phonemes from audio and affordability from prices, depending only on the data.

Vectorisation and Matrix Multiplication

The loop becomes one line

Loop over units (previous section)

```
for j in range(units):  
    z = np.dot(W[:, j], a_in) + b[j]  
    a_out[j] = g(z)
```

Vectorised, over units and examples

```
def dense(A_in, W, B):  
    Z = np.matmul(A_in, W) + B    # or A_in @ W + B  
    return g(Z)  
  
X = np.array([[ 1,  2],  
              [-1, -1]])        # 2 examples  
A1 = dense(X, W1, b1)           # shape (2, 3)  
A2 = dense(A1, W2, b2)          # shape (2, 1)  
# A2 = [[0.8094], [0.0562]]
```

Shapes

$$\underbrace{\mathbf{A}_{\text{in}}}_{(m, n_{\text{in}})} \underbrace{\mathbf{W}}_{(n_{\text{in}}, \text{units})} = \underbrace{\mathbf{Z}}_{(m, \text{units})}$$

row i of $\mathbf{Z}$: example i ; column j : unit j .

Why it matters

One matrix product replaces $m \times \text{units}$ dot products, and GPUs run matrix products in parallel. This is what makes large networks practical.

Broadcasting B

b has shape $(\text{units},)$; NumPy adds it to every row of $\mathbf{Z}$. A b of shape $(\text{units}, 1)$ either raises an error or — worse — broadcasts into the wrong values without complaint.

Building up

Vector · vector. $\mathbf{a} \cdot \mathbf{w} = a_1 w_1 + a_2 w_2 = \mathbf{a}^\top \mathbf{w}$

Row vector × matrix. $\mathbf{a}^\top \mathbf{W}$: one dot product per *column* of $\mathbf{W}$.

Matrix × matrix. $\mathbf{A}\mathbf{W}$: one dot product per (row of $\mathbf{A}$, column of $\mathbf{W}$) pair.

Rule

If $\mathbf{A}$ is $m \times n$ and $\mathbf{B}$ is $n \times p$, then $\mathbf{AB}$ is $m \times p$ and

$$(\mathbf{AB})_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

The *inner* dimensions must match.

Worked example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 5 & 0 & 1 \\ -1 & 2 & 0 \end{bmatrix}$$

$(2 \times 2)(2 \times 3) \rightarrow (2 \times 3)$:

$$\mathbf{AB} = \begin{bmatrix} 3 & 4 & 1 \\ 11 & 8 & 3 \end{bmatrix}$$

e.g. $(\mathbf{AB})_{21} = \text{row 2 of } \mathbf{A} \cdot \text{column 1 of } \mathbf{B}$
 $= 3 \cdot 5 + 4 \cdot (-1) = 11.$

Transpose

$\mathbf{A}^\top$ swaps rows and columns: $\mathbf{A}^\top = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$. In

NumPy, $\mathbf{A.T}$.

Order matters: $\mathbf{BA}$ is $(2 \times 3)(2 \times 2)$ — undefined here.

$$\begin{aligned}\mathbf{XW}^{[1]} + \mathbf{b}^{[1]} &= \begin{bmatrix} 1 & 2 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix} + [-1, 1, 2] \\ &= \begin{bmatrix} 5 & 5 & -7 \\ -3 & -1 & 1 \end{bmatrix} + [-1, 1, 2] = \begin{bmatrix} 4 & 6 & -5 \\ -4 & 0 & 3 \end{bmatrix} = \mathbf{Z}^{[1]} \\ \mathbf{A}^{[1]} &= g(\mathbf{Z}^{[1]}) = \begin{bmatrix} 0.9820 & 0.9975 & 0.0067 \\ 0.0180 & 0.5000 & 0.9526 \end{bmatrix} \\ \mathbf{A}^{[2]} &= g(\mathbf{A}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}) = g\left(\begin{bmatrix} 1.4464 \\ -2.8217 \end{bmatrix}\right) = \begin{bmatrix} 0.8094 \\ 0.0562 \end{bmatrix}\end{aligned}$$

Check the agreement

Row 1 of $\mathbf{Z}^{[1]}$ is the column of z values from the Part 1 table; row 2 is the second example. The loop and the matrix product compute the same numbers — confirm it with `np.allclose`.

Shapes along the way

$(2, 2)(2, 3) \rightarrow (2, 3)$, then $(2, 3)(3, 1) \rightarrow (2, 1)$.

The number of rows — examples — never changes; the number of columns is the width of the current layer.

Wrap-up and Assessment

What to remember

- Dense(units, activation) is one layer; Sequential chains them; predict runs forward propagation.
- Data is a 2-D matrix: one row per example, one column per feature.
- From scratch: dense loops over units; sequential loops over layers.
- Store unit weights as the columns of $\mathbf{W}$, of shape $(n_{\text{in}}, \text{units})$.
- Vectorised: $\mathbf{Z} = \mathbf{A}_{\text{in}}\mathbf{W} + \mathbf{b}$, $\mathbf{A}_{\text{out}} = g(\mathbf{Z})$.
- ANI is not AGI.

Formulas

$$\mathbf{A}^{[l]} = g(\mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}), \quad \mathbf{A}^{[0]} = \mathbf{X}$$
$$(m \times n)(n \times p) \rightarrow (m \times p)$$

Next

Week 5: training — the loss, compile and fit, activation functions beyond the sigmoid, multiclass output with softmax, and back-propagation.

1. Explain what `Dense(units=3, activation='sigmoid')` represents, and why its weights do not exist until the layer is first called. [5]
2. Explain why the coffee-roasting data cannot be classified well by one logistic unit but can be by a $2 \rightarrow 3 \rightarrow 1$ network. [6]
3. State the shapes of `np.array([5, 3])`, `np.array([[5, 3]])` and `np.array([[5], [3]])`, and say which one a TensorFlow layer expects for a single example with two features. [5]
4. What is a `tf.Tensor`? How do you convert it to a NumPy array, and why would you? [4]
5. Explain the roles of `Sequential`, `compile`, `fit` and `predict`. Which of them involves only forward propagation? [5]
6. Write the function `dense(a_in, W, b)` using a loop over units. Explain why the weights of unit j are `W[:, j]`. [6]

7. Distinguish ANI from AGI. Explain why rapid progress in the first does not by itself imply progress in the second. [5]
8. State the “one learning algorithm” hypothesis and describe one piece of evidence offered for it. What would it mean for AGI if it were true? [5]
9. State the rule for the shape of a matrix product and the formula for $(\mathbf{AB})_{ij}$. Compute $\mathbf{AB}$ for $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 1 & -1 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 1 & 3 \\ 4 & 2 \end{bmatrix}$. [6]
10. A layer receives $\mathbf{A}_{\text{in}}$ of shape $(100, 64)$ and has 25 units. Give the shapes of $\mathbf{W}$, $\mathbf{b}$, $\mathbf{Z}$ and $\mathbf{A}_{\text{out}}$. [5]
11. Using the matrix form, compute $\mathbf{Z}^{[1]}$ and $\mathbf{A}^{[2]}$ for the worked network with $\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 2 & 1 \end{bmatrix}$. [8]
12. Explain why a vectorised forward pass is much faster than nested loops, even though it performs the same arithmetic. [5]

1. **Design a verification harness.** You have written `dense` and `sequential` in NumPy, and a colleague built the same network in TensorFlow. Design a procedure that proves the two compute identical predictions, including how you copy weights across and the tolerance you accept. [12]
2. **Design for batch inference.** A mobile-banking app must score 2 million transactions overnight with a $30 \rightarrow 16 \rightarrow 8 \rightarrow 1$ network. Design the inference code: batch size, memory per batch, and the shape of every intermediate matrix. Justify your batch size. [12]
3. **Design a shape-bug detector.** Students' forward-propagation code often fails because of row/column confusion or a b of the wrong shape. Design a set of `assert` statements to place in `dense` that catch these bugs early, with an error message for each. [10]
4. **Design a claim.** A start-up says its product “thinks like a human radiologist”. Rewrite the claim as a testable statement, and design the evaluation that would support it. [10]

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§10.2** — multilayer neural networks, written with weight matrices $\mathbf{W}_1$, $\mathbf{W}_2$ and the layer-by-layer computation of activations.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§5.1** — feed-forward network functions, including absorbing the bias into the weights; **Appendix C** — properties of matrices.

Lab (CSE 816, Week 4): coffee roasting in TensorFlow; coffee roasting in NumPy; practice lab on neural networks for binary classification (digits 0 vs. 1).

Practice quizzes: *TensorFlow implementation*; *Neural network implementation in Python*.